

Bootstrap 5

Edoardo Lo Iacono

Contents

1	Introduzione a bootstrap	3
1.1	Installazione	3
1.1.1	CDN (Content Delivery Network)	3
1.1.2	Download locale	4
1.1.3	Installazione tramite pacchetto Node (NPM)	4
2	Breakpoints	4
2.1	Tabella dei Breakpoints	4
3	Container	5
3.1	Tipologie di Container	5
3.2	Comportamento Responsivo	5
4	Grid system	5
4.1	Struttura di base (.row e .col)	6
4.2	Griglia Responsiva e Dimensionamento	6
4.2.1	Esempio di Colonne Miste	6
4.3	Allineamento e Offset	7
4.4	Gutter (Spaziatura)	7
4.5	Riposizionamento orizzontale delle celle	7
5	Tipografia	8

1 Introduzione a bootstrap

Bootstrap è un **framework** open source per la creazione di siti e webapp. Utilizza un approccio di tipo **mobile-first** per creare pagine responsive ovvero in grado di adattarsi alle dimensioni dello schermo su cui si trovano. Bootstrap è stato sviluppato da Mark Otto e Jacob Thornton per creare twitter ed è poi stato successivamente reso open source nel 2011.

In altre parole Bootstrap è un insieme di regole che ci aiutano a lavorare meglio e più velocemente in particolar modo in riferimento al nostro css.

1.1 Installazione

Possiamo installare ed utilizzare bootstrap nei nostri progetti in diversi modi, vediamo di seguito i principali:

1.1.1 CDN (Content Delivery Network)

Il **CDN** permette di collegare Bootstrap al proprio sito senza dover scaricare nulla, basta incollare un link nell'header del file HTML. per usare bootstrap in questa maniera sarà necessario:

- Inserire il CSS dentro il tag `<head>`
- Inserire il file js appena prima della chiusura del tag `</body>`

CSS: <https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css>

JS: <https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js>

(link presi dal sito ufficiale di [bootstrap](#))

```
1 <!doctype html>
2 <html lang="en">
3
4 <head>
5   <meta charset="utf-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1">
7   <title>Bootstrap demo</title>
8   <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel
9     = "stylesheet"
10     integrity="sha384-sRI14kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB"
11     crossorigin="anonymous">
12 </head>
13
14 <body>
15   <h1>Hello, world!</h1>
16   <script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.
17     js"
18     integrity="sha384-FKyoEForCGlyvwx9Hj09JcYn3nv7wiPV1z7YYwJrWVcXK/BmnVDxM+D2scQbITxI"
19     crossorigin="anonymous"></script>
20 </body>
21 </html>
```

1.1.2 Download locale

Per poter utilizzare bootstrap anche in assenza di una connessione ad internet (necessaria se importato tramite CDN) possiamo scaricare alla seguente pagina: <https://getbootstrap.com/docs/5.3/getting-started/download/> il pacchetto con CSS e JS, il quale va estratto nella cartella del progetto e collegato mediante i richiami:

```
1 <link rel="stylesheet" href="css/bootstrap.min.css">
2 <script src="js/bootstrap.bundle.min.js"></script>
```

1.1.3 Installazione tramite pacchetto Node (NPM)

Se si vuole utilizzare bootstrap in un progetto che contenga node è sufficiente usare un gestore di pacchetti come **npm** e da terminale, all'interno della cartella di lavoro andare ad installare il pacchetto bootstrap tramite il comando:

```
1 npm i bootstrap@5.3.8
```

(In data 08/05/2026, l'edizione di bootstrap è 5.3.8, questa cambierà con il tempo, per la versione aggiornata fare riferimento al sito ufficiale!) Una volta installato sarà importabile nel file js tramite:

```
1 import 'bootstrap/dist/css/bootstrap.min.css';
```

2 Breakpoints

[Documentazione ufficiale](#)

I breakpoint sono i **punti di interruzione** su cui si basa il design responsivo, permettono infatti di identificare sei dimensioni principali riferite ai sei principali tipi di dispositivo:

2.1 Tabella dei Breakpoints

Dimensione	Sigla	Larghezza schermo	Dispositivo
Extra small		$< 576px$	Smartphone (verticale)
Small	sm	$\geq 576px$	Smartphone (orizzontale)
Medium	md	$\geq 768px$	Tablet
Large	lg	$\geq 992px$	Laptop
Extra Large	xl	$\geq 1200px$	Desktop standard
Extra Extra Large	xxl	$\geq 1400px$	Monitor grandi

I breakpoint ci serviranno in combinazione con le **media-query**: funzioni css che indicano come comportarsi in base alla larghezza o all'altezza del dispositivo.

Per poter utilizzare al meglio bootstrap è necessario importare all'inizio del file HTML il seguente meta-tag:

```
1 <meta name="viewport" content="width=device-width, initial-scale=1">
```

il quale dice di impostare la larghezza della pagina web uguale a quella del dispositivo ed imposta il livello di zoom iniziale ad 1:1

3 Container

[Documentazione ufficiale](#)

I container sono il componente fondamentale di bootstrap, permettono infatti di avere un design responsivo. I container non sono altro che dei contenitori che si adattano in base alle esigenze specificate, rappresentano infatti la base del **grid system** e consentono al loro interno di creare righe e colonne.

3.1 Tipologie di Container

Esistono 3 tipi di contenitore:

- **.container** = Assegna al contenitore una larghezza predefinita, si faccia riferimento alla prima riga della tabella sottostante
- **.container-{breakpoint}** = Rappresenta una via di mezzo tra .container e .container-fluid, imposta come larghezza 100% a tutti i terminali più piccoli rispetto a quello specificato ed in seguito aggiunge una larghezza predefinita.
- **.container-fluid** = Mette come larghezza sempre il 100% dello spazio disponibile

3.2 Comportamento Responsivo

Per una maggiore chiarezza si faccia riferimento alla seguente tabella presa dal sito ufficiale di bootstrap:

	Extra small <576px	Small ≥576px	Medium ≥768px	Large ≥992px	X-Large ≥1200px	XX-Large ≥1400px
.container	100%	540px	720px	960px	1140px	1320px
.container-sm	100%	540px	720px	960px	1140px	1320px
.container-md	100%	100%	720px	960px	1140px	1320px
.container-lg	100%	100%	100%	960px	1140px	1320px
.container-xl	100%	100%	100%	100%	1140px	1320px
.container-xxl	100%	100%	100%	100%	100%	1320px
.container-fluid	100%	100%	100%	100%	100%	100%

4 Grid system

[Documentazione ufficiale](#)

Il Grid system o sistema a griglia è l'elemento più importante ed innovativo di bootstrap, come abbiamo già detto in precedenza questo è possibile anche grazie alla presenza di Breakpoints e soprattutto container, visti nei paragrafi precedenti.

Il sistema a griglie di bootstrap non si basa sul già presente grid system del css ma su flexbox

4.1 Struttura di base (.row e .col)

La griglie in Bootstrap usano una serie di contenitori particolari chiamati `.row` e `.col` i quali permettono di allinare nella maniera desiderata il contenuto. Il funzionamento è pressoché analogo a quello delle tabelle in HTML (non a caso un tempo l'allineamento degli elementi veniva fatto proprio mediante l'ausilio di tabelle). Avremo quindi uno schema del tipo (`.container` → `.row` → `.col`):

```
1 <div class="container text-center">
2   <div class="row">
3     <div class="col">
4       Column
5     </div>
6     <div class="col">
7       Column
8     </div>
9     <div class="col">
10      Column
11    </div>
12  </div>
13 </div>
```

4.2 Griglia Responsiva e Dimensionamento

Analogamente a come abbiamo visto con i contenitori, possiamo inserire una larghezza dinamica in base al terminale andando ad aggiungere alla classe `.col` il breakpoint a cui facciamo riferimento. **Non** **tiamo** **oltre** **come** **bootstrap** **imposti** **automaticamente** **una** **struttura** **a** **12** **colonne**, possiamo andare a giocare con questa impostazione modificando la dimensione di esse ed andando a specificare quante colonne dedicare, se queste non vengono specificate viene fatto in automatico il calcolo 12/numero di colonne presenti. (nell'esempio precedente sarebbe analogo scrivere `class="col-4"`). Questa dimensione può anche essere specificata in base alla dimensione del dispositivo andando a specificare: `col-{breakpoint}-{dimensione}`.

4.2.1 Esempio di Colonne Miste

Per comprendere meglio questo concetto vediamo l'esempio di seguito:

```
1 <div class="container">
2   <div class="row">
3     <div class="col col-md-10" style="background-color: lightcyan;">Colonna 1</div>
4     <div class="col col-md-2" style="background-color: lightgoldenrodyellow;">Colonna
5       2</div>
6   </div>
</div>
```

Avendo impostato al div principale semplicemente la classe `.container`, esso perderà la larghezza 100% nei dispositivi più piccoli e successivamente una larghezza automatica in base alla dimensione, come spiegato in precedenza. con `col-md-*` stiamo specificando che quando la dimensione del dispositivo è $\geq 768\text{px}$ le due colonne prendono una sezione di spazio differente, quella color ciano occuperà

10/12 dello spazio mentre quella gialla 2/12, al contrario quando la dimensione del dispositivo è $\leq 768\text{px}$, entrambe le colonne avranno la stessa dimensione che equivale a `col-6`

4.3 Allineamento e Offset

`row row-cols-{breakpoint}-*` → Permette di specificare il numero di colonne che si vogliono vedere in una riga, è di nuovo possibile variare questa proprietà in base alla dimensione del dispositivo specificandolo tramite breakpoint.

Possiamo ancora allineare le nostre colonne con la classe `align-items-*` che ci permette di allinearle in alto, al centro o in basso rispetto ad una row oppure, con `align-self-*` di lavorare con la singola colonna, è inoltre possibile impostare un **offset**, ovvero il numero di colonne vuote davanti alla colonna interessata tramite il comando `col-{breakpoint}-offset-*`

4.4 Gutter (Spaziatura)

Il **gutter** serve per indicare lo **spazio tra le colonne** in maniera visiva, senza l'uso del paddingo, può essere applicato solamente insieme alla classe `.row`, ogni `.row` ha un gutter orizzontale (spazio tra le colonne) ed uno verticale (spazio tra le righe). Le classi disponibili sono le seguenti:

- `g-0` Nessun gutter (sia orizzontale che verticale)
- `g-1 ... g-5` Gutter crescente (sia orizzontale che verticale)
- `gx-0 ... gx-5` Gutter orizzontale specifico
- `gy-0 ... gy-5` Gutter verticale specifico

4.5 Riposizionamento orizzontale delle celle

```
1 <div class="container text-center">
2   <div class="row justify-content-start">
3     <div class="col-4">
4       One of two columns
5     </div>
6     <div class="col-4">
7       One of two columns
8     </div>
9   </div>
```

grazie alla classe `justify-content-*` possiamo posizionare le celle in orizzontale, i valori che possiamo inserire sono:

- `justify-content-start` → Situazione di default, allinea tutte le colonne verso sinistra lasciando lo spazio vuoto sulla destra.
- `justify-content-end` → Allinea tutte le colonne verso destra lasciando lo spazio vuoto sulla sinistra.
- `justify-content-center` → Centra le colonne orizzontalmente, lasciando lo stesso spazio vuoto sia a sinistra che a destra.

- `justify-content-between` → Posiziona la prima colonna all'inizio e l'ultima alla fine della riga, distribuendo uniformemente lo spazio vuoto tra le colonne.
- `justify-content-around` → Distribuisce lo spazio attorno ad ogni colonna; ogni elemento avrà spazio sia prima che dopo, ma lo spazio ai bordi esterni sarà circa la metà rispetto a quello tra due colonne.
- `justify-content-evenly` → Distribuisce lo spazio in modo perfettamente uniforme: lo spazio prima, dopo e tra tutte le colonne sarà uguale.

5 Tipografia

Parliamo di tipografia in relazione agli elementi ed alle classi che vanno a modificare il contenuto testuale; ci accorgiamo di aver inserito correttamente bootstrap all'interno del nostro progetto proprio guardando il testo, come esso cambia dinamicamente di dimensione, font e spaziatura.